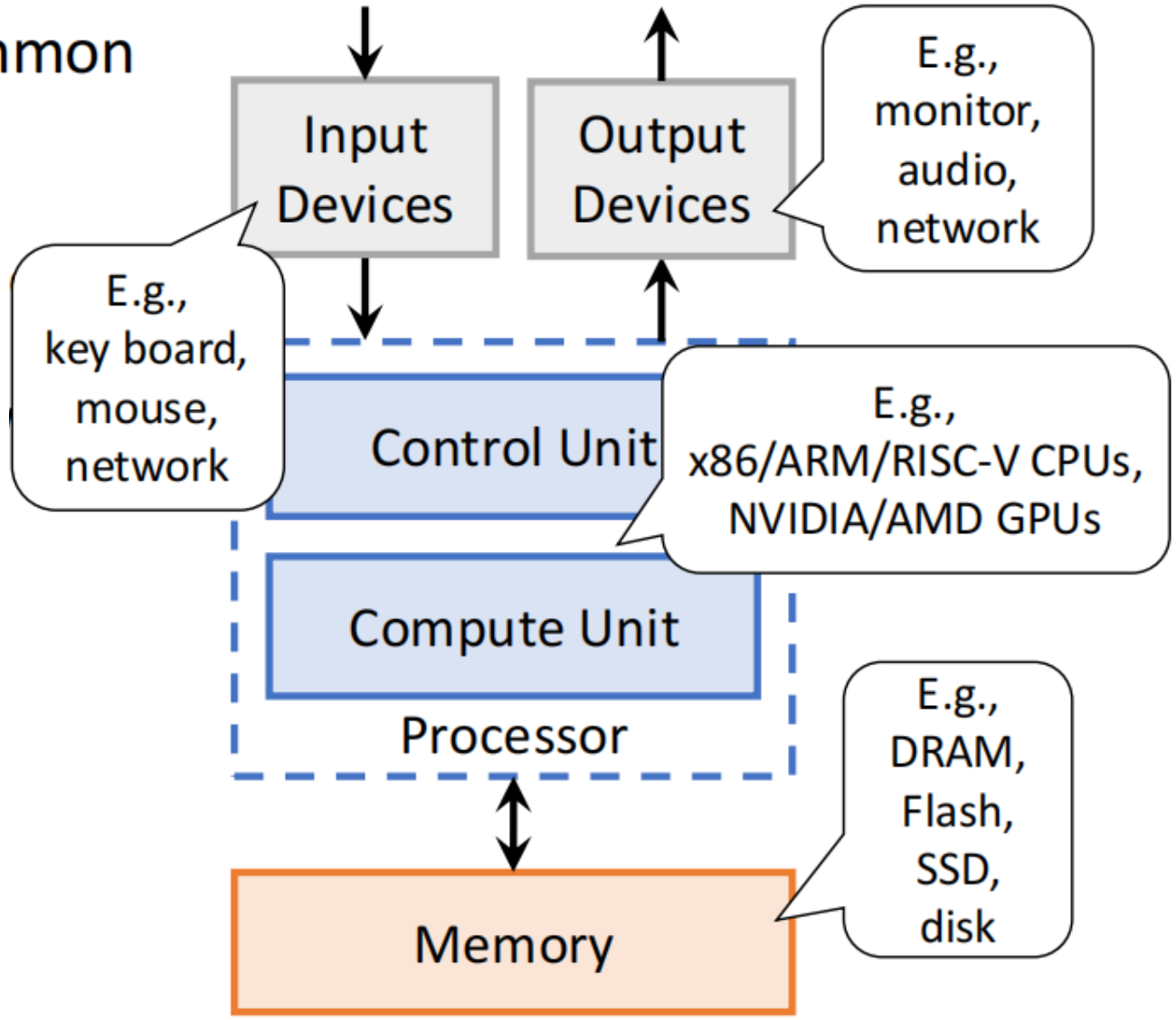
Programming in the LLM Era

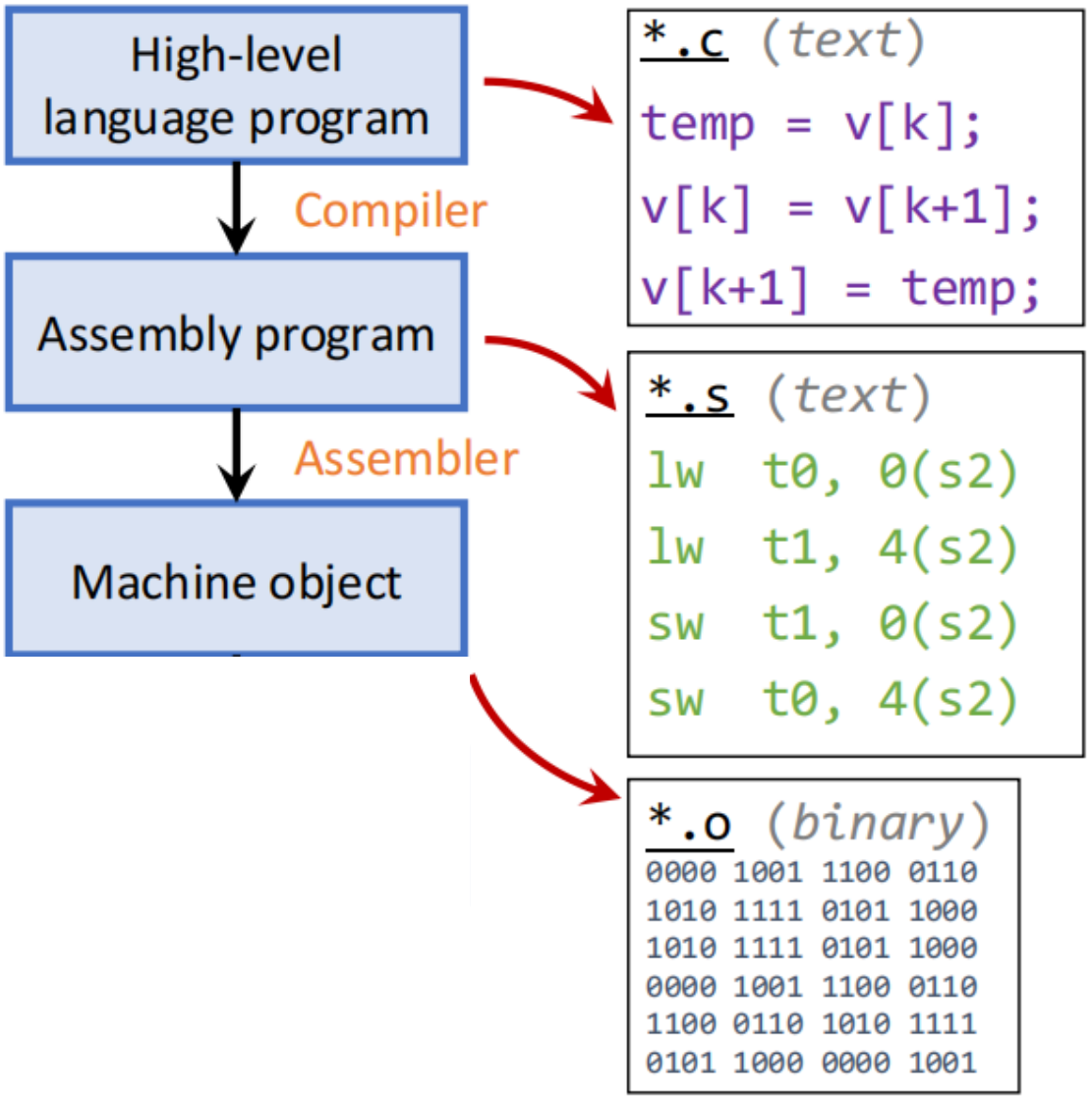
Von Neumann Architecture (from Prof. Gao's Lecture)

- Von Neumann Architecture: a common abstraction for typical systems



- Key point: separate processor and memory

Code Compilation and Translation (from Prof. Gao's Lecture)



- Machine code: the byte sequence that encodes program instructions
  - Actual 0's and 1's stored in the memory
- Assembly code: text representation of machine code
  - Human-readable instruction sequence

LLM is better and better at programming...

Recreating League of Legends (LOL)



LLM is better and better at programming...

Recreating CSGO



LLM is better and better at programming...

Recreating Minecraft



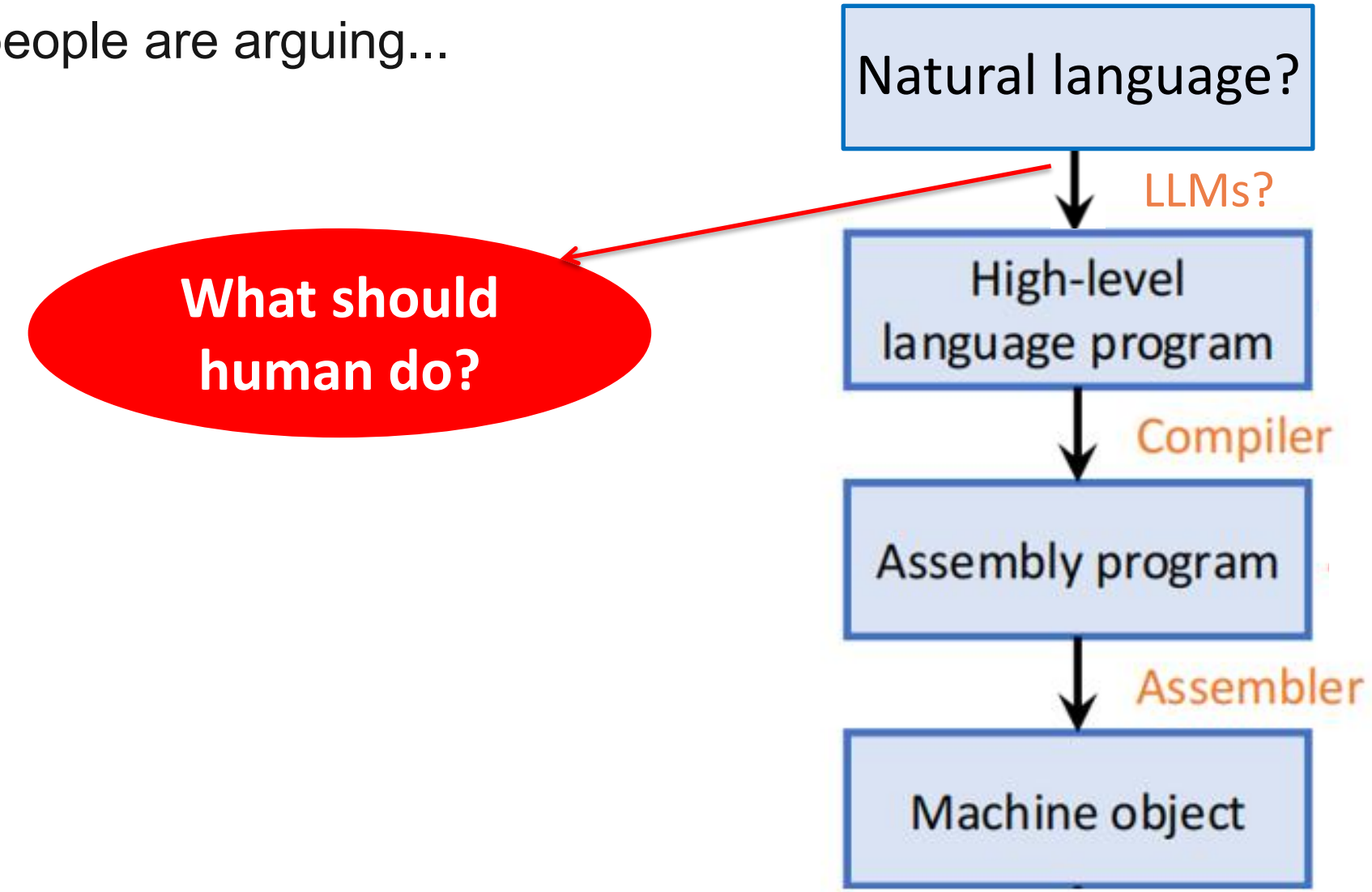
LLM is better and better at programming...

3D Modeling using Blender



An Emerging Idea: A Fourth Level?

Some people are arguing...



From a Real-World Problem to a Working Program

The process can be roughly divided into five steps.

Discover the problem › **Determine** what to do › **Implement** the code › **Verify** the result › **Deploy** in practice

A working program requires more than code generation.

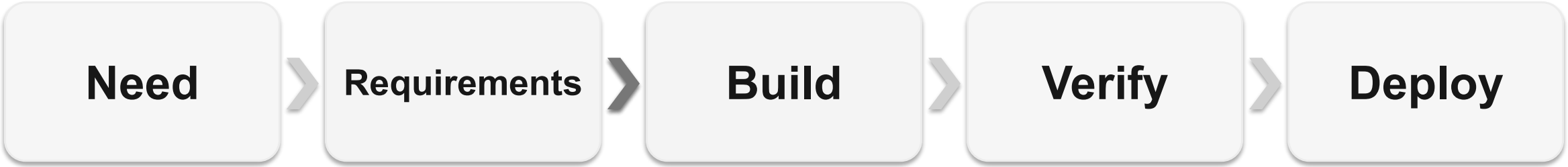
How Much Can LLMs Replace?



Need → Requirements **MEDIUM**

AI can propose requirements, but AI can’t make sure they are correct.

How Much Can LLMs Replace?



Requirements → Implementation **HIGH**

With clear requirements, AI can generate and revise substantial amounts of code.

How Much Can LLMs Replace?



Implementation → Verification **MEDIUM**

AI can produce tests and find bugs, but independent validation is still essential.

How Much Can LLMs Replace?



Verification → Deployment **MEDIUM**

AI can automate deployment tasks, but reliability and security still need oversight.

Where Is Human Judgment Still Necessary?

01

Define the goal

Choose clear success criteria.

02

Review the code

Spot assumptions and side effects.

03

Check the results

Test failures and edge cases.

AI can still assist at these steps but people are also necessary.

Define the Goal

What’s Wrong?

REAL-WORLD NEED

A school must choose one winner for an academic award.

PROMPT TO THE AI

“Read the grade sheet and select the best student.”

What is the “best”?

Semantic Ambiguity

Alice

Math 98 English 60 Physics 65

Bob

Math 88 English 90 Physics 91

Highest math score → Alice

Highest average → Bob

Reason: Prompts have different definitions.

What’s Wrong?

REAL-WORLD NEED

A teacher wants to rank students by their exam scores.

PROMPT TO THE AI

“Sort the spreadsheet by exam score in descending order.”

Sort what? Scores or students?

Lack of Implicit Invariants

Input

Name	Grade
Alice	60
Bob	80
Charlie	100



Output

Name	Grade
Alice	100
Bob	80
Charlie	60

Reason: The name and grade are not properly paired.

What’s Wrong?

REAL-WORLD NEED

Keep your thesis recoverable if the original disappears.

PROMPT TO THE AI

“Back up a/b, c/d, and e/f into the same backup folder, backup/.”

What is “backup”?

Implementation Ambiguity

ORIGINAL FILE

a/b, c/d, e/f

IN BACKUP/

a symbolic link for each file

not a second copy

Original deleted

Broken link

Reason: A symbolic link (soft link) is mistakenly treated as a backup.

What’s Wrong?

REAL-WORLD NEED

Number every trip photo without losing any existing file.

PROMPT TO THE AI

“Rename all images sequentially, starting with 1.jpg.”

How about the input?

Unsafe Intermediate States



Behavior differs across platforms and file APIs.

Reason: Filename collisions may cause data loss.

What's Wrong?

REAL-WORLD NEED

You have taken over a navigation application. It balances travel time, road types, tolls, and each driver's preferences. But it's quite slow.

PROMPT TO THE AI

"Optimize the route planner to run at least 100 times faster."

But at what cost?

Constraint Violation

BEFORE

The navigation app is slow but users like it.

AFTER

The navigation app directed users onto a very narrow road, causing users' strong dissatisfaction.

Reason: The change violates the original constraints.

What’s Wrong?

REAL-WORLD NEED

Checkout must recover when the payment reply is lost.

PROMPT TO THE AI

> **“If a payment request times out, retry it automatically.”**

Correctness?

Failure Semantics

01

Send $100

payment request

02

Server

charges $100

03

Reply

is lost

04

Retry without

idempotency key

Without duplicate-request protection, a second charge is possible.

> **Reason:** only “retry” is not enough.

Define the Goal

Common missing requirements are listed as follows.

- Define success
- Preserve constraints
- Consider the process
- Specify failure behavior

SKILL 02 / 03

Review the Code

Example 1

Requirements: A ticket is free for children aged 12 or younger, half price for seniors aged 65 or older.

Code:

```
def ticket_price(age):
    if age < 12:
        return 0
    elif age >= 65:
        return 5
    return 10
```

Example 2

Requirements: Return the three highest scores without modifying the original list.

Code:

```
def top_three(scores):
    scores.sort(reverse=True)
    return scores[:3]
```

This is an in-place sorting

Example 3

Requirements: Return all students whose score is at least 90.

Code:

```
def excellent_students(records):
    result = []
    for name, score in records:
        if score >= 90:
            result.append(name)
        return result
```

return too early

Example 4

Requirements: Sort exam records by score, from highest to lowest.

Code:

```
def rank_students(records):
    names = [name for name, score in records]
    scores = [score for name, score in records]

    scores.sort(reverse=True)

    return list(zip(names, scores))
```

corresponding relation is broken

SKILL 03 / 03

Check the Results

What Is Program Testing?

The Duck Test

If it looks like a duck, sounds like a duck, and acts like a duck, it is probably a duck.

Testing checks observable behavior against what we expect. **“The key to writing working software is developing good tests.”** (Source: Stanford CS106B — Testing)

Example

Requirements: Export students scoring at least 60, sorted by score. Preserve the original data.

Code:

```python
def export_report(src, dst):
    rows = open(src).readlines()
    students = []
    for row in rows[1:]:
        name, score = row.split(",")
        score = int(score)
        if score > 60:
            if name not in students:
                students.append(
                    (name, score)
                )
    students.sort(
        key=lambda x: -x[1]
    )
    with open(dst, "w") as f:
        for name, score in students:
            f.write(
                f"{name},{score}\n"
            )

```

Correctness

Requirements: Export students scoring at least 60, sorted by score. Preserve the original data.

Test case: 60 must be a good case.

Input: name,score | Alice,60

Expected: Alice,60 should appear in the report.

Actual: Alice is excluded; the report is empty.

Correctness

Problem: The comparison excludes the boundary value 60.

```python
def export_report(src, dst):
    rows = open(src).readlines()
    students = []
    for row in rows[1:]:
        name, score = row.split(",")
        score = int(score)
        if score > 60:
            if name not in students:
                students.append(
                    (name, score)
                )
    students.sort(
        key=lambda x: -x[1]
    )
    with open(dst, "w") as f:
        for name, score in students:
            f.write(
                f"{name},{score}\n"
            )

```

Robustness

Requirements: Export students scoring at least 60, sorted by score. Preserve the original data.

Test case: An input row contains an invalid score.

Input: name,score | Alice,85 | Bob,N/A

Expected: Reject or report the invalid row in a defined way.

Actual: ValueError; the program stops.

Robustness

Problem: The code assumes that every score is a valid integer.

```python
def export_report(src, dst):
    rows = open(src).readlines()
    students = []
    for row in rows[1:]:
        name, score = row.split(",")
        score = int(score)
        if score > 60:
            if name not in students:
                students.append(
                    (name, score)
                )
    students.sort(
        key=lambda x: -x[1]
    )
    with open(dst, "w") as f:
        for name, score in students:
            f.write(
                f"{name},{score}\n"
            )

```

Reliability

Requirements: Export students scoring at least 60, sorted by score. Preserve the original data.

Test case: A disk error occurs while writing the report.

Input: Existing report.csv + simulated write failure

Expected: The existing valid report remains recoverable.

Actual: The old report may be truncated or partly overwritten.

Reliability

Problem: Writing directly to the destination risks partial output.

```python
def export_report(src, dst):
    rows = open(src).readlines()
    students = []
    for row in rows[1:]:
        name, score = row.split(",")
        score = int(score)
        if score > 60:
            if name not in students:
                students.append(
                    (name, score)
                )
    students.sort(
        key=lambda x: -x[1]
    )
    with open(dst, "w") as f:
        for name, score in students:
            f.write(
                f"{name},{score}\n"
            )

```

Performance

Requirements: Export students scoring at least 60, sorted by score. Preserve the original data.

Test case: The input grows from 1,000 to 100,000 rows.

Input: CSV files of increasing sizes

Expected: Runtime should remain practical at the target scale.

Actual: The repeated list search can become very slow.

Performance

Problem: Checking membership in a growing list can take quadratic time.

```python
def export_report(src, dst):
    rows = open(src).readlines()
    students = []
    for row in rows[1:]:
        name, score = row.split(",")
        score = int(score)
        if score > 60:
            if name not in students:
                students.append(
                    (name, score)
                )
    students.sort(
        key=lambda x: -x[1]
    )
    with open(dst, "w") as f:
        for name, score in students:
            f.write(
                f"{name},{score}\n"
            )

```

THE NEXT QUESTION

What Will Humans Still Do?

Of Course It’s an Open Question

But I believe the first one always matter.

01

Define the goal

Choose clear success criteria.

02

Review the code

Spot assumptions and side effects.

03

Check the results

Test failures and edge cases.